

TideWise Deployment Guide

Making Your Dashboard Live & Shareable

This guide covers multiple options for deploying TideWise so anyone can access your live dashboard via a URL (e.g., <https://tidewise-sa.up.railway.app>).

Option 1: Railway.app (RECOMMENDED)

Best for: Quick deployment, free tier, automatic HTTPS, custom domains

Cost: FREE for hobby projects (500 hours/month = ~20 days runtime)

Step-by-Step:

1. Create Railway Account

- Go to <https://railway.app>
- Click "Start a New Project"
- Sign up with GitHub (recommended)

2. Connect Your GitHub Repo

- Push your code to GitHub:

```
cd tidewise-prototype
git init
git add .
git commit -m "TideWise - Coastal Risk Alert System"
git remote add origin https://github.com/YOUR_USERNAME/tidewise.git
git push -u origin main
```

3. Deploy on Railway

- In Railway dashboard, click "New Project"
- Select "Deploy from GitHub repo"
- Choose your **tidewise** repository
- Railway auto-detects Python and uses:
 - **requirements.txt** for dependencies
 - **Procfile** to know how to start app
 - **runtime.txt** for Python version

4. Add Environment Variables (if needed)

- In Railway project settings → Variables:

```
PORT=5000 (auto-set by Railway)
DEBUG=False
```

5. Get Your Public URL

- Railway automatically generates: <https://tidewise-production.up.railway.app>
- Under Settings → Domains, you can:
 - Use the generated subdomain
 - Add custom domain (e.g., tidewise.co.za)

6. Share Your Dashboard!

- Copy the URL: <https://tidewise-production.up.railway.app>
- Share on WhatsApp, email, presentations
- Live dashboard updates automatically every 30 seconds

Railway Pros:

- ✓ Free tier (500 hours/month)
- ✓ Automatic HTTPS
- ✓ Auto-deploys on git push (CI/CD)
- ✓ Easy environment variables
- ✓ Built-in monitoring

Railway Cons:

- ⚠ Sleeps after 30 min inactivity (first visitor waits ~30 seconds)
- ⚠ 500 hours/month = needs paid plan (\$5/mo) if running 24/7

◇ Option 2: Render.com (Also Great)

Best for: Always-on free tier, no sleep

Cost: FREE (limited resources) or \$7/month (better performance)

Quick Deploy:

1. **Go to** <https://render.com>
2. **Create Web Service** → Connect GitHub repo
3. **Settings:**
 - Build Command: `pip install -r requirements.txt`
 - Start Command: `python web_dashboard.py`
 - Environment: Python 3
4. **Deploy** - Get URL like <https://tidewise-sa.onrender.com>

Render Pros:

- ☑ Free tier doesn't sleep (slower but always available)
- ☑ Automatic HTTPS
- ☑ Easy setup

Render Cons:

- ⚠ Free tier is SLOW (can take 30-60 seconds to load page)
- ⚠ Limited to 750 hours/month on free tier

☁ Option 3: Google Cloud Run (Free Tier)

Best for: Professional deployment, scales automatically

Cost: FREE up to 2 million requests/month

Deploy Steps:

1. Install Google Cloud CLI:

```
# Windows (PowerShell as Admin)
(New-Object
Net.WebClient).DownloadFile("https://dl.google.com/dl/cloudsdk/channels/r
apid/GoogleCloudSDKInstaller.exe",
"$env:Temp\GoogleCloudSDKInstaller.exe")
& $env:Temp\GoogleCloudSDKInstaller.exe
```

2. Create Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "web_dashboard.py"]
```

3. Deploy to Cloud Run:

```
gcloud run deploy tidewise \
  --source . \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated
```

4. Get URL: <https://tidewise-abc123-uc.a.run.app>

Cloud Run Pros:

- ✓ Generous free tier
- ✓ Scales to zero (no cost when idle)
- ✓ Professional-grade infrastructure
- ✓ Custom domains easy

Cloud Run Cons:

- ⚠ More complex setup
 - ⚠ Requires Docker knowledge
-

Option 4: Heroku (Classic Choice)

Best for: Legacy deployments (many tutorials available)

Cost: FREE tier removed in 2022 → \$5/month minimum

Deploy Steps:

1. Install Heroku CLI:

```
# Windows
winget install Heroku.HerokuCLI
```

2. Deploy:

```
heroku login
heroku create tidewise-sa
git push heroku main
```

3. Get URL: <https://tidewise-sa.herokuapp.com>

Heroku Pros:

- ✓ Simple deployment
- ✓ Mature platform
- ✓ Lots of documentation

Heroku Cons:

- ✗ No free tier anymore (\$5/month minimum)
 - ⚠ More expensive than alternatives
-

Option 5: Self-Host with Ngrok (Quick Demo)

Best for: Temporary sharing during presentations/demos

Cost: FREE (with ngrok subdomain)

Quick Setup:

1. **Download ngrok:** <https://ngrok.com/download>

2. **Run TideWise locally:**

```
cd tidewise-prototype
python web_dashboard.py
```

3. **In another terminal:**

```
ngrok http 5000
```

4. **Get temporary URL:** <https://abc123.ngrok.io>

- Share this URL - anyone can access your dashboard
- **Valid only while your computer is running**

Ngrok Pros:

- ☒ Instant sharing (no deployment)
- ☒ Free tier available
- ☒ Great for demos/testing

Ngrok Cons:

- ⚠ Requires your computer to stay on
- ⚠ URL changes each time you restart
- ⚠ Not suitable for permanent deployment

Option 6: Azure App Service (Enterprise)

Best for: If targeting South African government/municipalities

Cost: FREE tier (F1) or \$13/month (B1 - recommended)

Why Azure for SA Market:

- Microsoft has Azure datacenter in Johannesburg
- Lower latency for South African users
- Government procurement familiarity with Azure
- Can integrate with other Azure services (AI, databases, etc.)

Deploy Steps:

1. Install Azure CLI:

```
winget install Microsoft.AzureCLI
```

2. Login & Deploy:

```
az login  
az webapp up --name tidewise-sa --runtime "PYTHON:3.12" --location  
southafricanorth
```

3. Get URL: <https://tidewise-sa.azurewebsites.net>

Azure Pros:

- ☒ South African datacenter (fast for local users)
- ☒ Government-friendly (procurement)
- ☒ Enterprise features (monitoring, auto-scaling)
- ☒ Free tier available

Azure Cons:

-  More complex than Railway/Render
-  Free tier has limitations (60 min/day CPU time)

RECOMMENDATION FOR YOU

Based on your use case (sharing with coastal communities, potential hackathon demo, future business):

Start with Railway.app ☒

Why:

1. **Free & Fast** - 500 hours/month = plenty for demos
2. **Professional URL** - tidewise-sa.up.railway.app looks legit
3. **Easy to Share** - HTTPS, no setup required for users
4. **Auto-deploy** - Push to GitHub → automatically updates live site
5. **Upgrade Path** - Easy to switch to paid (\$5/mo) if needed

Next Steps:

1. Push to GitHub:

```
cd tidewise-prototype
git init
git add .
git commit -m "TideWise v1.0 - Live coastal risk monitoring"

# Create repo on github.com, then:
git remote add origin https://github.com/YOUR_USERNAME/tidewise.git
git push -u origin main
```

2. Deploy to Railway:

- Go to railway.app
- "New Project" → "Deploy from GitHub"
- Select your repo
- Wait 2-3 minutes for deployment
- Copy the URL: <https://tidewise-xxxxx.up.railway.app>

3. Share Your Dashboard:

- **WhatsApp:** "Check out TideWise - live coastal flood alerts for Port Elizabeth: <https://tidewise-xxxxx.up.railway.app>"
- **Email to Ocean Hub:** "Proof of concept ready for review: [link]"
- **Presentation:** Show live dashboard during pitch

4. Monitor Usage:

- Railway dashboard shows requests/second, errors, logs
- See how many people are viewing your dashboard
- Check for any errors in production

Security Considerations

Before going live, consider:

1. Rate Limiting (prevent abuse):

```
pip install flask-limiter
```

Add to [web_dashboard.py](#):

```
from flask_limiter import Limiter
limiter = Limiter(app, key_func=lambda: request.remote_addr)

@app.route('/api/current')
@limiter.limit("60 per minute") # Max 60 requests/min per IP
```

```
def get_current_data():  
    ...
```

2. **Error Handling** (hide debug info):

- Already set `debug=False` in production ☒
- Add custom error pages for 404, 500 errors

3. **Monitoring** (know when it breaks):

- Use Railway's built-in monitoring
- Or add Sentry for error tracking:

```
pip install sentry-sdk[flask]
```

4. **Cost Alerts** (avoid surprise bills):

- Railway: Set usage alerts in dashboard
- Most platforms: Email when approaching free tier limits

Comparison Table

Platform	Cost	Speed	Ease	Uptime	SA Location
Railway ★	Free (500h)	Fast	Easy	99.9%	No (US/EU)
Render	Free (slower)	Medium	Easy	99.5%	No (US)
Cloud Run	Free (2M req)	Fast	Medium	99.95%	No (US/EU/Asia)
Azure	Free (60m/day)	Fast	Hard	99.95%	YES (Joburg)
Heroku	\$5/mo	Fast	Easy	99.9%	No (US/EU)
Ngrok	Free (temp)	Fast	Very Easy	N/A (local)	Your PC

What Happens After Deployment

Once live, your dashboard:

1. **Updates automatically** every 30 seconds with fresh NOAA data
2. **Accessible globally** - anyone with the URL can view
3. **Works on mobile** - responsive design adapts to phone screens
4. **Shows real-time alerts** - Green/Yellow/Orange/Red based on actual conditions
5. **Demonstrates capability** - proves technical feasibility to investors/partners

Example Share Message:

"🐼 TideWise is LIVE!

Real-time coastal flood risk monitoring for Port Elizabeth, South Africa.

🔗 **Dashboard:** <https://tidewise-sa.up.railway.app>

See live ocean conditions updated every 30 seconds:

- Atmospheric pressure from NOAA satellites
- Wave height & tide levels
- Risk assessment (Green/Orange/Red alerts)

Built for LoCO AUV | Ocean Hub Africa

Proof of concept - seeking pilot partners"

💡 Pro Tips

1. **Custom Domain** (looks more professional):

- Buy tidewise.co.za (~R150/year)
- Point to Railway URL in DNS settings
- Now share: <https://tidewise.co.za> instead of Railway subdomain

2. **QR Code** (for presentations):

- Generate QR code linking to your dashboard
- Print on posters/flyers for community meetings
- Instant mobile access

3. **Analytics** (track visitors):

- Add Google Analytics to [dashboard.html](#)
- See how many people visit, where they're from
- Measure community engagement

4. **Feedback Form** (improve based on user input):

- Add simple form to dashboard footer
- Ask: "Is this useful? What's missing?"
- Collect testimonials for business case

🚀 Ready to Deploy?

Fastest path (5 minutes):

1. Create GitHub account (if you don't have one)
2. Push your code to new repo
3. Sign up for Railway.app
4. Deploy from GitHub

5. Share your live URL!

Need help? Let me know which option you want to use and I'll guide you through the specific steps! 